

UNIVERSIDAD TECNOLÓGICA DE CHIHUAHUA

COMPREHENSIVE WEB DEVELOPMENT

IDGSBIS91N



TEACHER:

ING. RAMSES ARTURO REQUENA VELASCO

ELABORATED BY:

KARIM CARLOS

Design Patterns in a Web Application Prototype

9/20/2026

Full-Stack Web Development: Design Patterns Project Justification

Project Context and Problem Definition This prototype addresses the complex inventory and build-management needs of a custom car audio shop. Fine-tuning a car's sound system requires highly specific parts—such as configuring compact active subwoofers (e.g., the Kicker Hideaway 10"), matching them with graphic equalizers to achieve a precise V-shaped audio profile, and installing 1-DIN retractable screen head units. Managing these distinct, highly technical items in a unified software architecture requires robust object creation and data management protocols. Without a structured approach, the codebase would quickly become disorganized as new audio equipment types are added to the shop's catalog.

2. Technology Stack Selection The application is constructed utilizing Next.js (App Router), React, and TypeScript. Next.js was selected due to its integrated API routing framework, which allows both the front-end (Views) and back-end (Controllers and Models) to be developed in a single repository while strictly enforcing architectural boundaries. TypeScript provides static typing, which is fundamentally critical when defining strict interfaces for Creational design patterns, ensuring that an Equalizer object and a HeadUnit object share the necessary structural contracts.

3. Design Pattern Implementations and Justifications The prototype successfully implements three core design patterns to solve specific application requirements:

- **Singleton (State and Connection Management):** In a production environment, continuously establishing new database connections to read a user's audio build sheet introduces high latency and risks exhausting database connection pools. The Singleton pattern was implemented via the `DatabaseConnection` class to guarantee that the application shares a single, centralized instance of the data cache. By making the constructor private, the system strictly controls the instantiation process, preventing any module from bypassing the connection manager.
- **Factory (Component Instantiation):** Car audio installations are highly modular. The application code handling the addition of new components should not need to know the specific class constructors for every piece of hardware on the market. The `AudioComponentFactory` class isolates this responsibility. When a user adds a subwoofer, the factory determines the correct object to instantiate and applies the necessary default

parameters (like wattage and frequency response profiles). This adheres to the Open/Closed Principle: expanding the catalog to include DSPs (Digital Signal Processors) requires adding a single class and one switch-case, rather than rewriting the application's core logic.

- MVC - Model-View-Controller (Architectural Organization): To ensure long-term maintainability, the application architecture is strictly partitioned. The Controller acts as a validation gateway, processing incoming HTTP requests and ensuring valid hardware types are passed. The Model houses the core business logic, orchestrating data flow between the Singleton database and the Factory[cite: 1]. Finally, the View components in the React front-end are completely decoupled from network logic and data manipulation; they solely consume the processed data properties and render the user interface. This separation ensures that any future changes to the visual interface will have zero impact on the underlying business rules.